

Assignment 4

DATA 260

Name: Kazi Samina Maraj Mumu

SJSU ID: 018324098

Email: Kazisaminamaraj.mumu@sjsu.edu

Date: 09/28/2026

Municipal Transit Incident Hub

Hardware: Apple MacBook Pro with Apple M5 chip and 24 GB memory

Local Model: qwen3:8b via Ollama

Tagged Commit Hash: d3e9cd3

SID4: 4098

Port: 8498

Database: s4098_rel

Prefix: s4098

Seed: 4098

Verification seed: 264098

Overview

This project builds on the previous transit incident application by adding MySQL database storage, user authentication, protected CRUD operations, a React frontend, performance testing, database indexing, and a retrieval-augmented generation (RAG) experiment.

The application is designed for managing municipal transit incidents. After logging in, users can view existing incidents and create, update, or delete records. The backend was developed with FastAPI and SQLAlchemy, while the frontend was built with React and Vite.

The React frontend runs at:

<http://localhost:5173>

The FastAPI backend runs at:

<http://localhost:8498>

Authentication and Protected CRUD

The application includes email/password authentication and session-cookie protection.

The following operations were tested successfully:

- User registration
- User login
- User logout
- /auth/me
- Protected incident listing
- Creating an incident
- Updating an incident
- Deleting an incident

For frontend testing, I created an incident titled HW4 Frontend Test. The incident was successfully created, updated to HW4 Frontend Updated, and deleted.

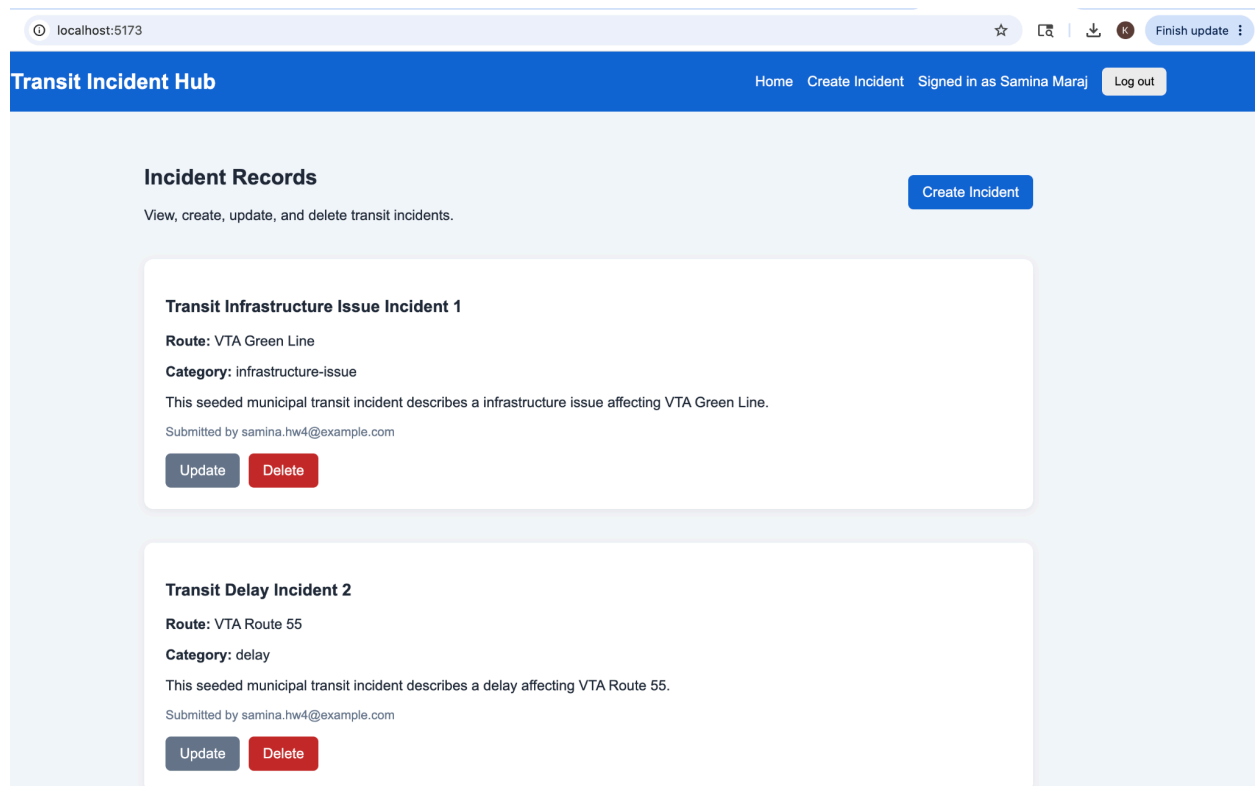


Figure 1. Authenticated React Home page displaying transit incidents and CRUD controls.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8498/api/hw4/incidents' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "incidentTitle": "HW4 Final Evidence Test",
    "routeLine": "VTA Test Route",
    "submitterEmail": "samina.hw4@example.com",
    "description": "Temporary incident created for final HW4 evidence.",
    "category": "delay",
    "termsAccepted": true
  }'
```

Request URL

http://localhost:8498/api/hw4/incidents

Server response

Code	Details
201	Response body <pre>{ "id": 5003, "incidentTitle": "HW4 Final Evidence Test", "routeLine": "VTA Test Route", "submitterEmail": "samina.hw4@example.com", "description": "Temporary incident created for final HW4 evidence.", "category": "delay", "termsAccepted": true, "submissionDate": "2026-09-28T19:13:13", "related": [] }</pre> Download Response headers <pre>access-control-allow-credentials: true content-length: 283 content-type: application/json date: Mon, 28 Sep 2026 19:13:12 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response	No links

Figure 2. Successful creation of temporary incident 5003 through the protected POST endpoint.

Responses

Code	Description	Links
200	Successful Response	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "incidentTitle": "string", "routeLine": "string", "submitterEmail": "user@example.com", "description": "string", "category": "string", "termsAccepted": true, "submissionDate": "string", "related": [] }</pre>		
422	Validation Error	No links
Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>		

localhost:8498/docs#/HW4/update_incident_api_hw4_incidents__incident_id__put

☆ | 🔍 | ⬇️ | 🔄 | Finish update

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8498/api/hw4/incidents/5002' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "incidentTitle": "HW4 Frontend Updated",
    "routeLine": "VIA Test Route",
    "submitterEmail": "samina.hw4@example.com",
    "description": "Temporary incident created to test the React CRUD workflow.",
    "category": "delay",
    "termsAccepted": true
  }'
```

Request URL

http://localhost:8498/api/hw4/incidents/5002

Server response

Code	Details
200	Response body <pre>{ "id": 5002, "incidentTitle": "HW4 Frontend Updated", "routeLine": "VIA Test Route", "submitterEmail": "samina.hw4@example.com", "description": "Temporary incident created to test the React CRUD workflow.", "category": "delay", "termsAccepted": true, "submissionDate": "2026-09-26T05:20:05", "related": [] }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 289 content-type: application/json date: Sat, 26 Sep 2026 05:35:19 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Figure 3. Successful update of incident 5002 through the protected API.

Curl

```
curl -X 'GET' \
  'http://localhost:8498/api/hw4/incidents?skip=0&limit=100' \
  -H 'accept: */*'
```

Request URL

http://localhost:8498/api/hw4/incidents?skip=0&limit=100

Server response

Code	Details
200	Response body <pre>{ "id": 2, "incidentTitle": "Transit Infrastructure Issue Incident 1", "routeLine": "VTA Green Line", "submitterEmail": "samina.hw4@example.com", "description": "This seeded municipal transit incident describes a infrastructure issue affecting VTA Green Line.", "category": "infrastructure-issue", "termsAccepted": true, "submissionDate": "2025-09-26T04:16:36", "related": [{ "id": 1, "related_type": "follow_up", "related_text": "Related test note 1 for incident 2." }] }, { "id": 3, "incidentTitle": "Transit Delay Incident 2", "routeLine": "VTA Route 55", "submitterEmail": "samina.hw4@example.com", "description": "This seeded municipal transit incident describes a delay affecting VTA Route 55.", "category": "delay" }</pre> Response headers <pre>content-length: 34415 content-type: application/json date: Mon, 28 Sep 2026 19:19:48 GMT server: uvicorn</pre>

Responses

Figure 4. Successful retrieval of all incidents through the protected GET endpoint.

incident_id * required
integer
(path)

5003

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8498/api/hw4/incidents/5003' \
  -H 'accept: */*'
```

Request URL

http://localhost:8498/api/hw4/incidents/5003

Server response

Code	Details
200	Response body <pre>{ "id": 5003, "incidentTitle": "HW4 Final Evidence Test", "routeLine": "VTA Test Route", "submitterEmail": "samina.hw4@example.com", "description": "Temporary incident created for final HW4 evidence.", "category": "delay", "termsAccepted": true, "submissionDate": "2026-09-28T19:13:13", "related": [] }</pre> Response headers

Figure 5. Successful retrieval of incident 5003 by ID through the protected GET endpoint.

Incident id required

integer

(path)

5003

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:8498/api/hw4/incidents/5003' \
-H 'accept: */*'
```

Request URL

```
http://localhost:8498/api/hw4/incidents/5003
```

Server response

Code	Details
200	Response body <pre>{ "message": "Incident deleted", "id": 5003 }</pre> Download Response headers <pre>access-control-allow-credentials: true content-length: 48 content-type: application/json date: Mon, 28 Sep 2026 19:22:50 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Figure 6. Successful deletion of temporary incident 5003 through the protected DELETE endpoint.

I tested the protected CRUD endpoints using temporary incident records. The screenshots show that creating, retrieving, updating, and deleting incidents all worked successfully. Since the tests were done separately, the incident IDs are different.

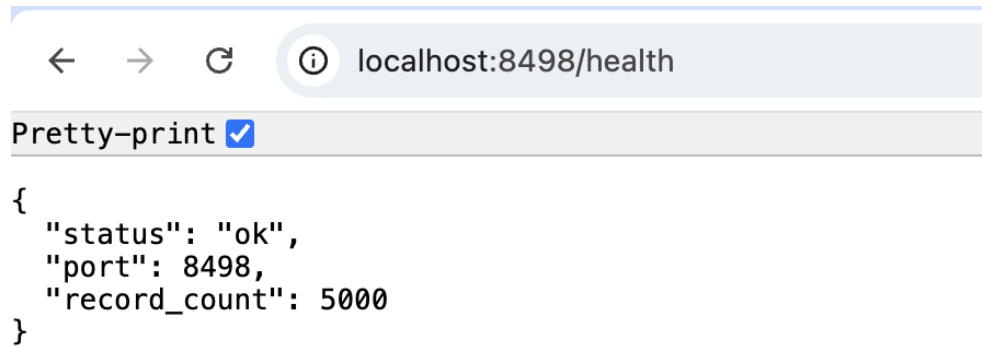
MySQL Persistence and Health Check

The database contains 5,000 seeded transit incident records.

The health endpoint returned:

```
{
  "status": "ok",
  "port": 8498,
  "record_count": 5000
}
```

This confirms that the application is connected to MySQL and is reading the record count from the database instead of using an in-memory list.

A screenshot of a web browser window. The address bar shows 'localhost:8498/health'. Below the address bar, there is a 'Pretty-print' checkbox which is checked. The main content area displays a JSON response: { "status": "ok", "port": 8498, "record_count": 5000 }.

```
{
  "status": "ok",
  "port": 8498,
  "record_count": 5000
}
```

Figure 7. Health endpoint confirming MySQL persistence with 5,000 transit incident records.

N+1 Query Benchmark

The benchmark tested two implementations:

- naive: performs an additional related-data query for each incident
- fixed: uses eager loading or a joined query to avoid repeated queries

Each configuration was tested 30 times for page sizes 10, 50, and 200, producing 180 total measurements.

Page size	Version	SQL statements/re quest	p50 ms	p95 ms	p99 ms
10	naive	13	5.301	7.174	9.310
10	fixed	4	2.742	4.234	11.572
50	naive	53	14.756	17.512	22.746
50	fixed	4	4.059	4.135	4.198
200	naive	203	51.222	61.845	66.399

200	fixed	4	11.463	13.144	21.862
-----	-------	---	--------	--------	--------

The naive implementation increases from 13 to 53 to 203 SQL statements as the page size increases. This happens because it performs one additional query for each incident.

The fixed implementation remains at four SQL statements per request. For page size 200, the fixed version reduced p50 latency from 51.222 ms to 11.463 ms, which is approximately a 77.6% reduction.

```
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % cat reports/hw04/raw/np1us1_summary.json
```

```
[
  {
    "version": "fixed",
    "page_size": 10,
    "requests": 30,
    "rows_per_response": 10,
    "sql_statements_per_request": 4.0,
    "p50_ms": 2.742,
    "p95_ms": 4.234,
    "p99_ms": 11.572
  },
  {
    "version": "fixed",
    "page_size": 50,
    "requests": 30,
    "rows_per_response": 50,
    "sql_statements_per_request": 4.0,
    "p50_ms": 4.059,
    "p95_ms": 4.135,
    "p99_ms": 4.198
  },
  {
    "version": "fixed",
    "page_size": 200,
    "requests": 30,
    "rows_per_response": 200,
    "sql_statements_per_request": 4.0,
    "p50_ms": 11.463,
    "p95_ms": 13.144,
    "p99_ms": 21.862
  },
  {
    "version": "naive",
    "page_size": 10,
    "requests": 30,
    "rows_per_response": 10,
    "sql_statements_per_request": 13.0,
    "p50_ms": 5.301,
    "p95_ms": 7.174,
    "p99_ms": 9.31
  },
  {
    "version": "naive",
    "page_size": 50,
    "requests": 30,
    "rows_per_response": 50,
    "sql_statements_per_request": 53.0,
    "p50_ms": 14.756,
    "p95_ms": 17.512,
    "p99_ms": 22.746
  },
  {
    "version": "naive",
    "page_size": 200,
    "requests": 30,
    "rows_per_response": 200,
    "sql_statements_per_request": 203.0,
    "p50_ms": 51.222,
    "p95_ms": 61.845,
    "p99_ms": 66.399
  }
]
```

```
]%
```

```
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 8. Complete N+1 benchmark output for the naive and fixed implementations across page sizes 10, 50, and 200

Index and EXPLAIN Evidence

An index was added to the incident category field:

ix_incidents_category

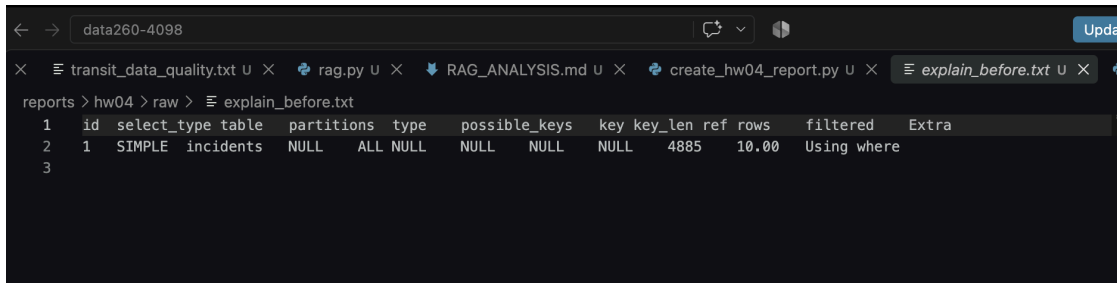
Before the index:

type=ALL
key=NULL
rows=4885

After the index:

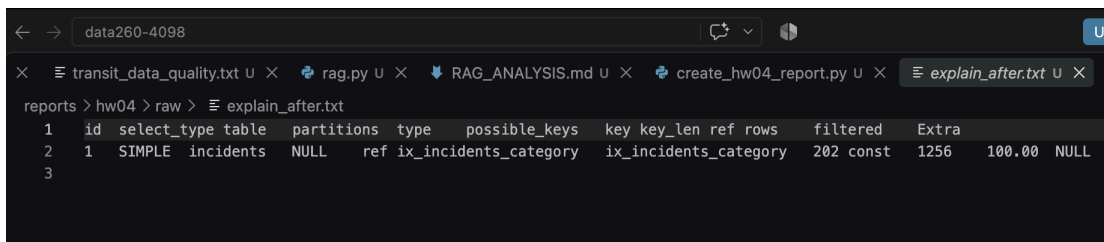
type=ref
key=ix_incidents_category
rows=1256

Before the index, MySQL used a full table scan. After the index was added, MySQL used an indexed lookup. The estimated number of rows examined decreased from 4,885 to 1,256.



```
data260-4098
reports > hw04 > raw > explain_before.txt
1 id select_type table partitions type possible_keys key key_len ref rows filtered Extra
2 1 SIMPLE incidents NULL ALL NULL NULL NULL 4885 10.00 Using where
3
```

Figure 9a. EXPLAIN output before adding the category index, showing a full table scan.



```
data260-4098
reports > hw04 > raw > explain_after.txt
1 id select_type table partitions type possible_keys key key_len ref rows filtered Extra
2 1 SIMPLE incidents NULL ref ix_incidents_category ix_incidents_category 202 const 1256 100.00 NULL
3
```

Figure 9b. EXPLAIN output after adding ix_incidents_category, showing an indexed lookup.

RAG Experiment

The RAG experiment used five documents related to transit safety and security incidents. The documents included public transit safety information and three smaller text documents about incident records, incident response, and data quality.

The documents were divided into 500-character chunks with 50-character overlap. I created embeddings using **all-MiniLM-L6-v2** and stored them in a FAISS vector index. The local Ollama model qwen3:8b generated the answers.

Three configurations were compared:

1. **No RAG:** The model answered without retrieved documents.
2. **Basic RAG:** Retrieved chunks were provided as context.
3. **Context-engineered RAG:** Retrieved chunks were labeled with source names, and the model was instructed to answer only from the provided context and refuse unsupported questions.

Six questions were tested across all three configurations, producing 18 comparison records. A k-sweep was also run with k values of 1, 3, and 5.

The first four questions were supported by the transit documents. The fifth question asked for the exact fare of VTA Route 55, and the sixth asked who won the 2025 World Series. These last two questions were outside the document collection and were used as refusal tests.

Context RAG correctly refused the unsupported fare and World Series questions. This showed safer behavior than unrestricted generation because it did not confidently invent answers from outside the documents.

The retrieval results also showed a limitation. Even after increasing the main retrieval size to 15 chunks, the system sometimes ranked the larger HTML and CSV documents above the smaller custom text documents. This caused the automatic retrieval metric to remain false for some supported questions. This result demonstrates that vector similarity alone is not always enough. A stronger system could combine embeddings with keyword search, metadata filtering, or reranking.

Overall, Context RAG was the safest configuration because it used document context, identified sources, and refused unsupported questions. The experiment also showed that retrieval quality and answer quality should be evaluated separately.

Five-document corpus

```
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % find reports/hw04/corpus -maxdepth 1 -type f -print
reports/hw04/corpus/transit_data_quality.txt
reports/hw04/corpus/transit_safety_basics.txt
reports/hw04/corpus/ntd_safety_security_overview.html
reports/hw04/corpus/fra_regulated_mode_major_security_events.csv
reports/hw04/corpus/transit_incident_response.txt
```

Evaluation table

```
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % cat reports/hw04/raw/evaluation_table.csv
question,configuration,correct_retrieval,correct_answer,grounded,refused_when_needed
Q1,no_rag,False,True,False,True
Q1,basic_rag,False,True,False,True
Q1,context_rag,False,True,False,True
Q2,no_rag,False,True,False,True
Q2,basic_rag,False,True,False,True
Q2,context_rag,False,False,False,True
Q3,no_rag,False,True,False,True
Q3,basic_rag,False,True,False,True
Q3,context_rag,False,True,False,True
Q4,no_rag,False,False,False,True
Q4,basic_rag,False,True,False,True
Q4,context_rag,False,True,False,True
Q5,no_rag,False,False,False,False
Q5,basic_rag,False,False,False,False
Q5,context_rag,False,True,True,True
Q6,no_rag,False,False,False,False
Q6,basic_rag,False,False,False,False
Q6,context_rag,False,True,True,True
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

k-sweep output

```
0 event description: [Source 4] (M= Injury details: [Source 3] and [Source 4])
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % ls reports/hw04/raw/*rag* reports/hw04/raw/k_sweep.jsonl reports/hw04/raw/evaluation_table.csv
reports/hw04/raw/evaluation_table.csv reports/hw04/raw/k_sweep.jsonl reports/hw04/raw/rag_comparison.jsonl
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 %
```

Figure 10. RAG corpus, comparison results, evaluation table, and k-sweep evidence.

Required Evidence Files

The following files were created:

reports/hw04/RUN_LOG.txt
reports/hw04/METRICS.md
reports/hw04/AI_USE.md
reports/hw04/verification.json
reports/hw04/RAG_ANALYSIS.md
reports/hw04/raw/nplus1_measurements.jsonl
reports/hw04/raw/nplus1_summary.json
reports/hw04/raw/explain_before.txt
reports/hw04/raw/explain_after.txt
reports/hw04/raw/retrieved_chunks.jsonl
reports/hw04/raw/rag_comparison.jsonl
reports/hw04/raw/k_sweep.jsonl
reports/hw04/raw/evaluation_table.csv
reports/hw04/report.pdf

Key Code Evidence

```
@app.get("/health")
def health(db: Session = Depends(get_db)) -> dict:
    return {
        "status": "ok",
        "port": PORT_BASE,
        "record_count": db.query(hw4_models.IncidentRecord).count(),
    }
```

Figure 11. FastAPI health endpoint reading the incident count from MySQL.

```
category: Mapped[str] = mapped_column(
    String(50),
    nullable=False,
    index=True,
)
```

Figure 12. SQLAlchemy model definition adding an index to the incident category field.

Conclusion

In this homework, I built a complete municipal transit incident application using React, FastAPI, and MySQL. I added user authentication and protected CRUD operations so users can safely create, view, update, and delete incident records.

The performance tests showed why avoiding N+1 queries is important. The fixed version used only four SQL statements per request, while the number of queries in the naive version increased significantly as the page size grew. Adding the category index also improved the database query plan by changing it from a full table scan to an indexed lookup.

Finally, the RAG experiment showed that using document context can make answers safer and more reliable. Context RAG correctly refused questions that were not supported by the transit documents. Overall, this homework helped me understand how frontend applications, APIs, databases, performance optimization, indexing, and document-based AI can work together in one system.